

System Architecture

> Relevant source files

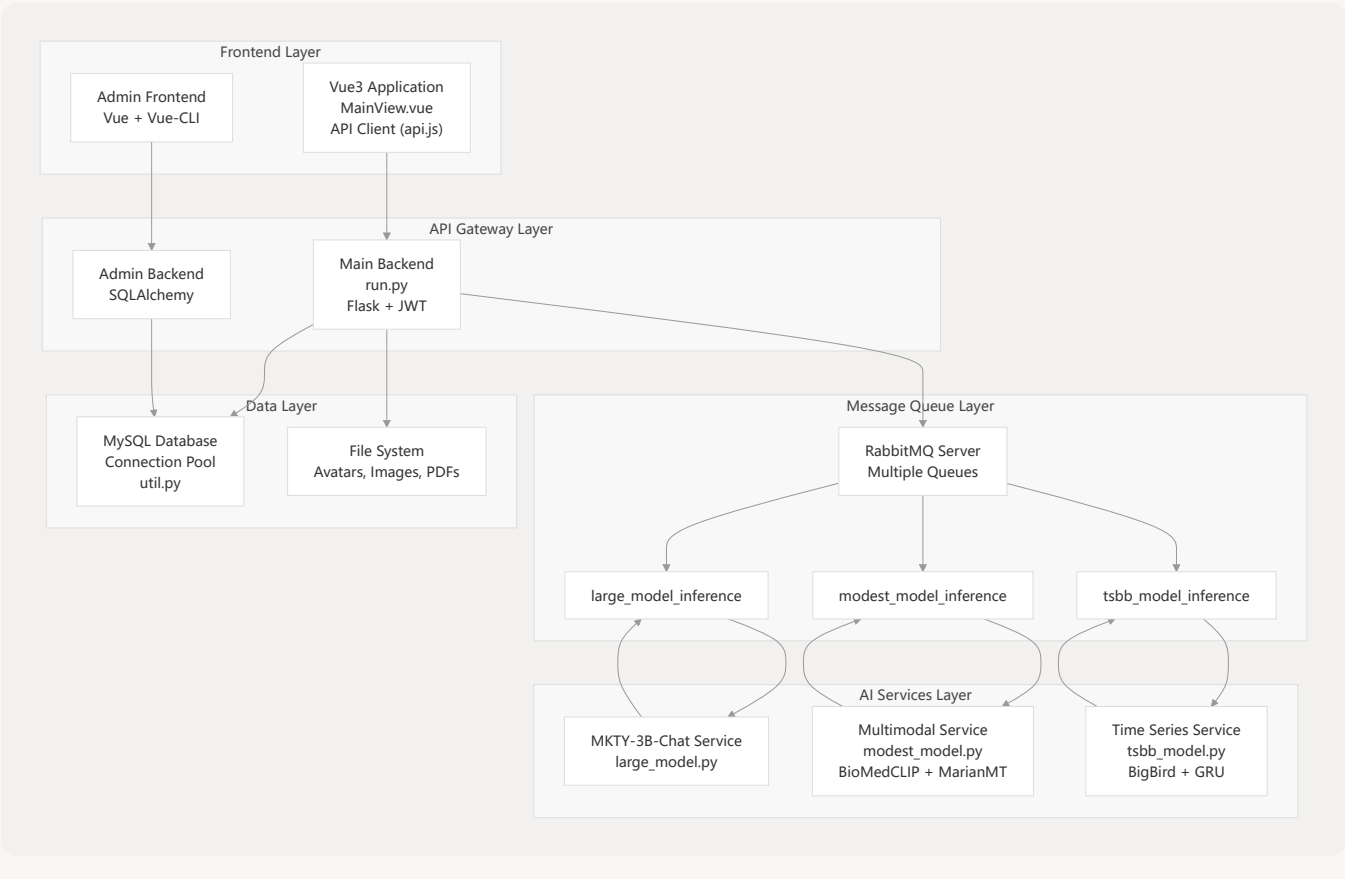
This document provides a comprehensive technical overview of the MKTY Smart Healthcare System architecture, including the distributed system design, component interactions, data flow patterns, and deployment structure. It covers the frontend Vue3 application, Python Flask backend services, AI model integration, message queue communication, and database management.

For information about specific AI models and machine learning implementations, see [AI & Machine Learning Services](#). For details about user interfaces and frontend components, see [Frontend Application](#). For administrative system architecture, see [Administration System](#).

Architecture Overview

The MKTY system follows a distributed microservices architecture with clear separation between frontend, backend business logic, AI services, and data storage layers. The system is designed to handle medical consultations, diagnostic assistance, forum discussions, and healthcare management workflows.

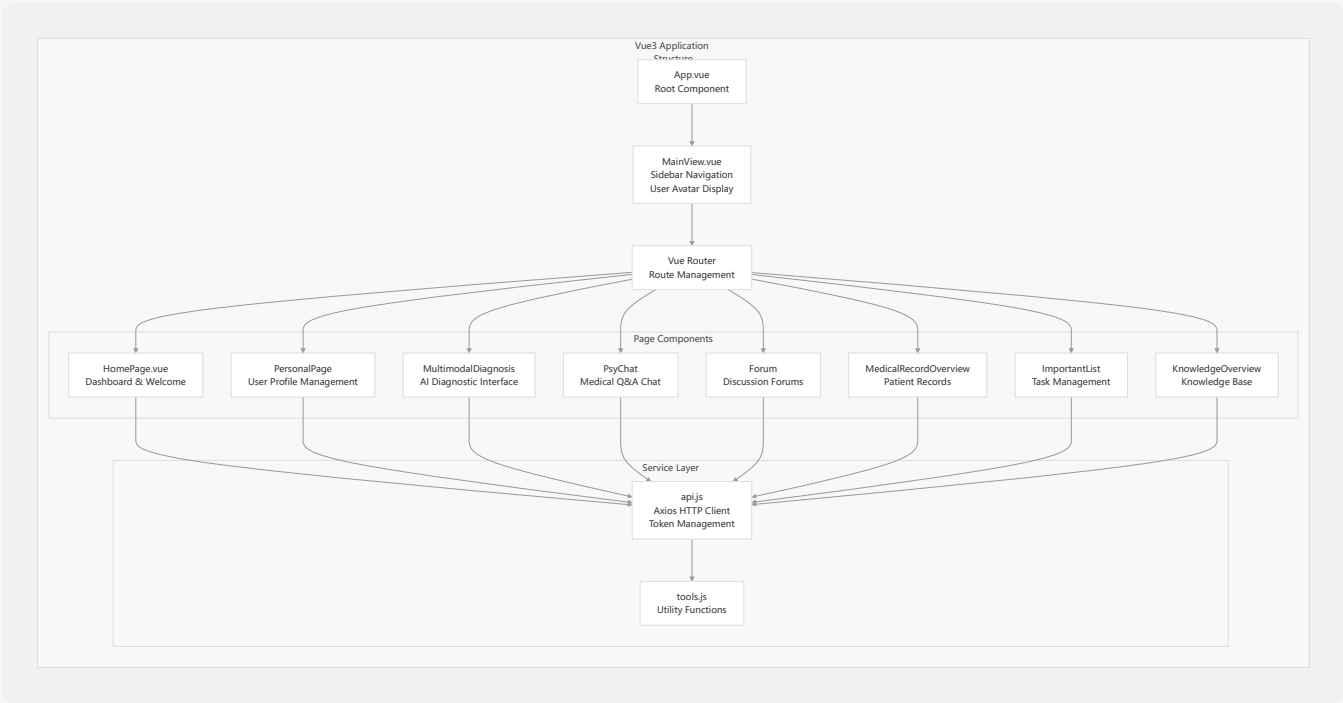
High-Level System Architecture



Frontend Architecture

The frontend is built as a Single Page Application using Vue3 with Element Plus UI components. The application follows a component-based architecture with centralized API management and routing.

Frontend Component Structure



API Client Architecture

The frontend communicates with backend services through a centralized API client that handles authentication, request/response processing, and error handling.

Function Category	API Functions	Authentication	Purpose
Authentication	loginVerification , register	None	User login and registration
User Management	getUserInfo , modifyUserInfo , getUserAvatar	JWT Required	Profile management
Medical Services	multimodalDiagnosisSubmitTask , llmInferenceSubmitTask	JWT Required	AI-powered medical services

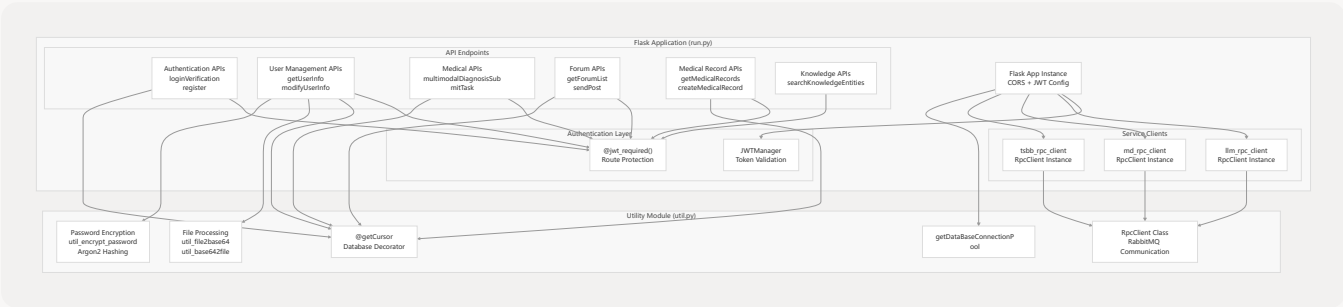
Function Category	API Functions	Authentication	Purpose
Medical Records	<code>getMedicalRecords</code> , <code>createMedicalRecord</code>	JWT Required	Patient record management
Forums	<code>getForumList</code> , <code>sendPost</code> , <code>getPostList</code>	JWT Required	Community discussions
Knowledge Base	<code>searchKnowledgeEntities</code> , <code>createKnowledgeEntity</code>	JWT Required	Knowledge management
Task Management	<code>getImportantList</code> , <code>addImportantItem</code>	JWT Required	Medical task scheduling

Sources: `frontend/src/api/api.js` 80-541

Backend Services Architecture

The backend consists of a main Flask application that serves as the API gateway, along with specialized utility modules for database management, file processing, and inter-service communication.

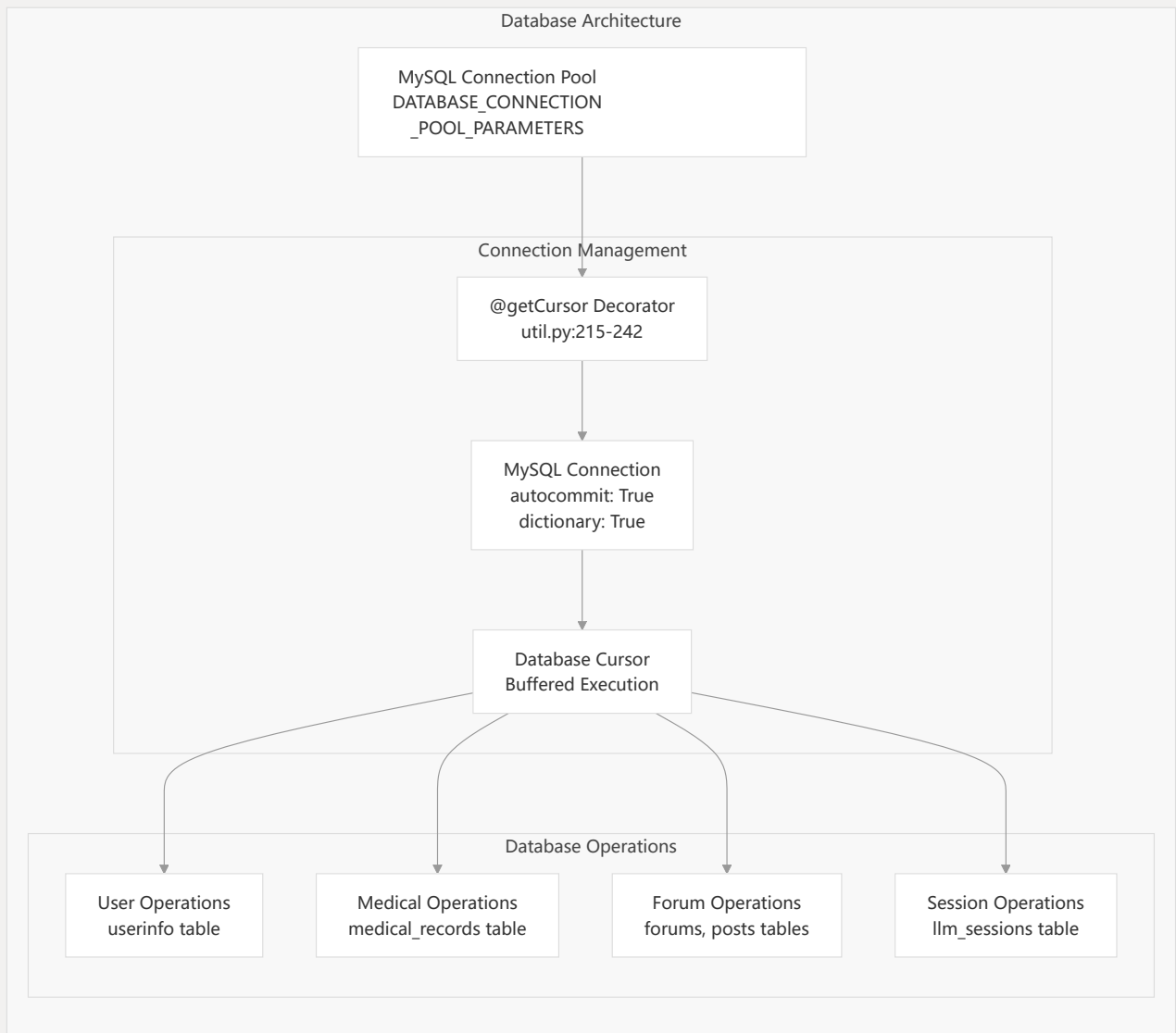
Main Backend Service Structure



Sources: `backend/run.py` 90-101 `backend/run.py` 86-88 `backend/util.py` 269-334 `backend/util.py` 215-242

Database Connection Management

The system uses MySQL with connection pooling for efficient database access. The `getCursor` decorator provides automatic connection management for API endpoints.

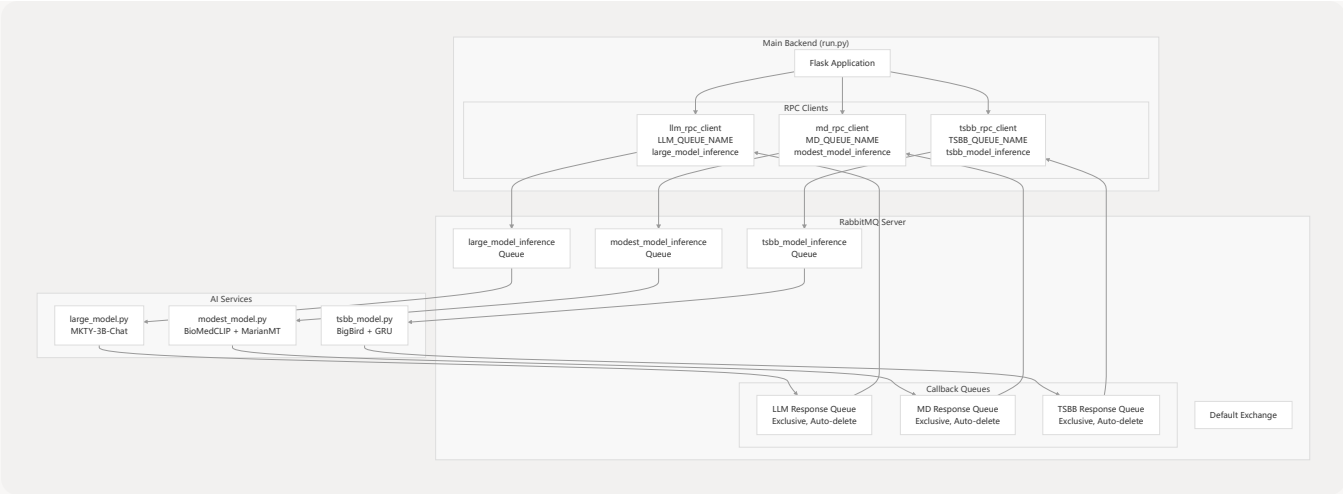


Sources: `backend/run.py` 68-75 `backend/util.py` 190-213 `backend/util.py` 215-242

Message Queue Architecture

The system uses RabbitMQ for asynchronous communication between the main backend and AI services. Each AI service has its own dedicated queue for task distribution and result collection.

RabbitMQ Communication Pattern



Sources: `backend/run.py` 50-67 `backend/run.py` 86-88 `backend/util.py` 269-334

RPC Client Implementation

The `RpcClient` class in `util.py` implements the RabbitMQ RPC pattern with correlation IDs for tracking async requests.

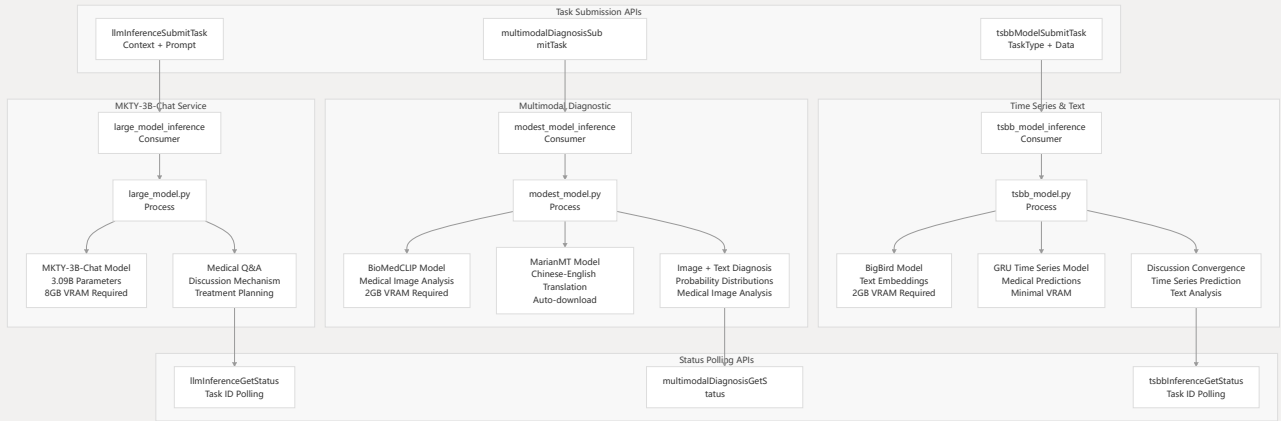
Method	Purpose	Parameters	Returns
<code>__init__</code>	Initialize RabbitMQ connection	<code>mq_connection_parameters</code> , <code>queue_name</code>	<code>RpcClient</code> instance
<code>call</code>	Submit async task	<code>data</code> (serializable object)	<code>corr_id</code> (task UUID)
<code>get_response</code>	Check task status	<code>corr_id</code> (task UUID)	Response data or None
<code>on_response</code>	Handle incoming responses	Callback parameters	Stores response in <code>self.responses</code>

Sources: `backend/util.py` 296-334

AI Services Integration

The system integrates three distinct AI services, each running as separate processes and communicating via RabbitMQ. Each service is optimized for specific medical AI tasks.

AI Service Architecture

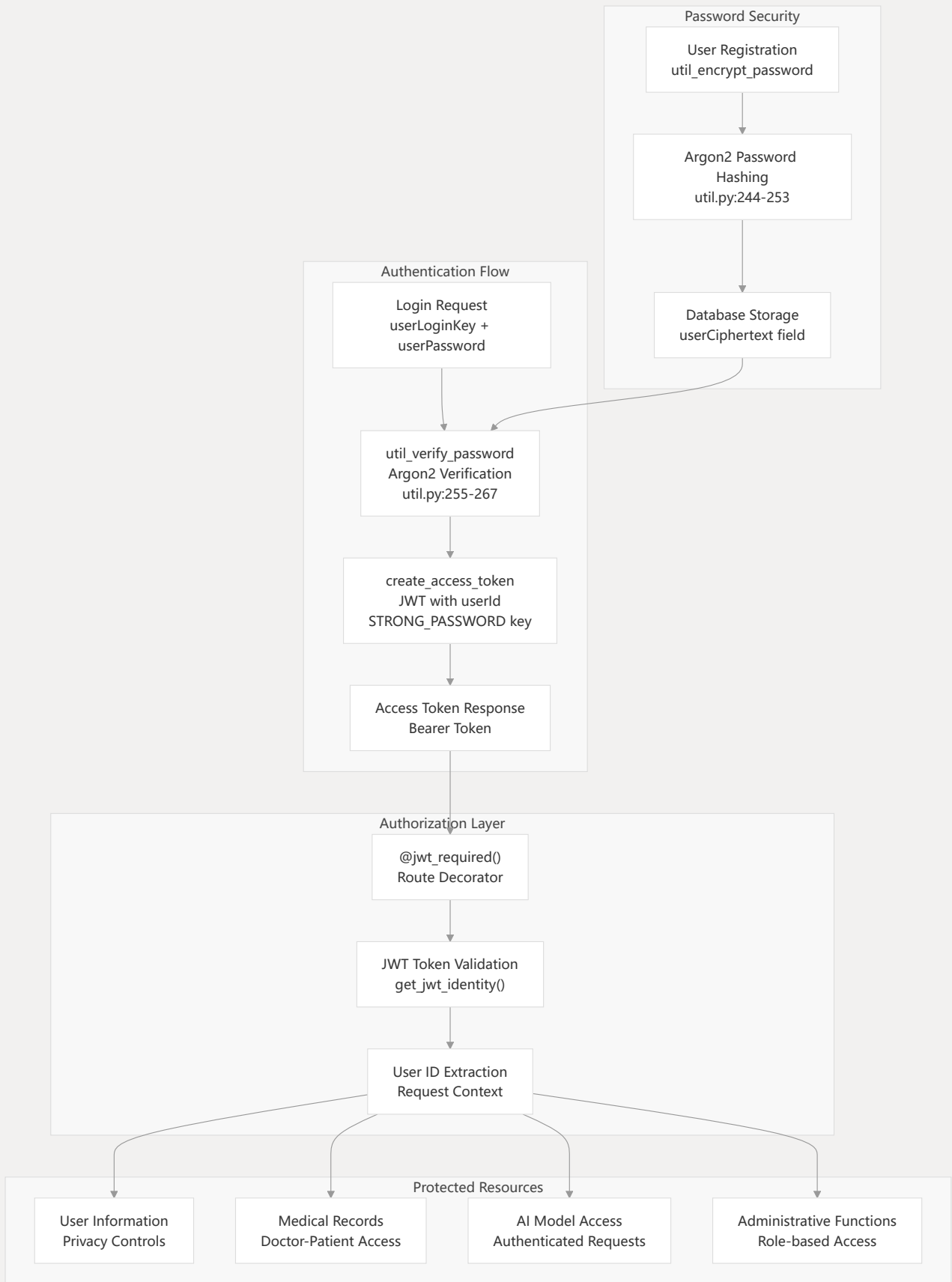


Sources: `README.md` 295-298 `backend/run.py` 262-277 `backend/run.py` 656-670
`backend/run.py` 1063-1077

Authentication and Security

The system implements JWT-based authentication with Argon2 password hashing and role-based access control for medical data protection.

Security Architecture



Sources: backend/run.py 118-157 backend/run.py 93-95 backend/util.py 244-267
backend/run.py 159-224

Data Flow Architecture

The system follows a clear data flow pattern from frontend user interactions through backend processing to AI service execution and result delivery.

End-to-End Data Flow

Frontend Layer

User Action
Vue Component



API Call
api.js axios request

